

```
# Step 1: Install Dependencies
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense, LSTM, Dropout
```

```
from sklearn.preprocessing import MinMaxScaler
```

```
import yfinance as yf
```

```
print("Libraries imported successfully. TensorFlow version:", tf.__version__)
```

```
# Fetch Google (GOOGL) Stock Data
```

```
print("Downloading Google stock data...")
```

```
data = yf.download('GOOGL', start='2010-01-01', end='2023-12-31')
```

```
# Preprocess for training
```

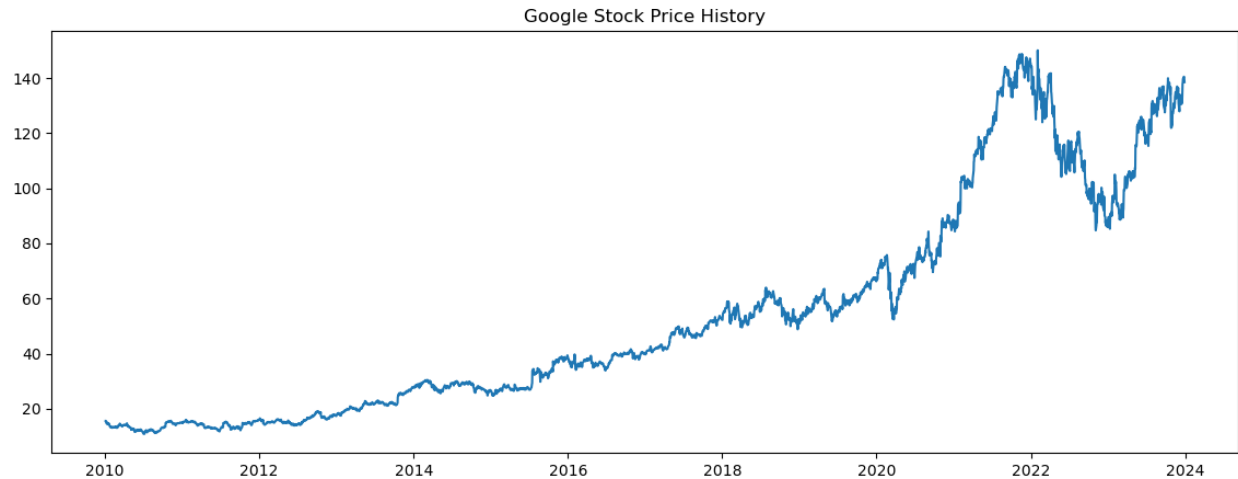
```
training_set = data.iloc[:, 0:1].values
```

```
plt.figure(figsize=(14, 5))
```

```
plt.plot(data['Open'])
```

```
plt.title('Google Stock Price History')
```

```
plt.show()
```



```
# Feature Scaling
```

```
sc = MinMaxScaler(feature_range=(0, 1))
```

```
training_set_scaled = sc.fit_transform(training_set)
```

```
# Create sliding window (60 days)
```

```
X_train = []
```

```
y_train = []
```

```
for i in range(60, len(training_set_scaled)):
```

```
    X_train.append(training_set_scaled[i-60:i, 0])
```

```
    y_train.append(training_set_scaled[i, 0])
```

```
X_train, y_train = np.array(X_train), np.array(y_train)
```

```
# Reshape for LSTM [samples, timesteps, features]
```

```
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
```

```
print(f"Data preprocessed. X_train shape: {X_train.shape}")
```

```
model = Sequential([
```

```
    LSTM(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1)),
```

```
Dropout(0.2),
LSTM(units=50, return_sequences=True),
Dropout(0.2),
LSTM(units=50),
Dropout(0.2),
Dense(units=1)
])
```

```
model.compile(optimizer='adam', loss='mean_squared_error')
print("Model summary:")
model.summary()
print("Training the RNN (20 epochs)... This may take a moment.")
model.fit(X_train, y_train, epochs=20, batch_size=32)
predicted_stock_price = model.predict(X_train)
predicted_stock_price = sc.inverse_transform(predicted_stock_price)
```

```
plt.figure(figsize=(14, 5))
plt.plot(training_set[60:], color='red', label='Real Price')
plt.plot(predicted_stock_price, color='blue', label='Predicted Price')
plt.title('Google Stock Price Prediction Result')
plt.legend()
plt.show()
```

Google Stock Price Prediction Result

